

# Become a Software Architect

---

I'm not exactly sure when I became a software architect. I do remember the first time someone else called me one. We were at an important client meeting and someone asked a tough technical question. The project manager chimed in: "Michael is the architect on this project. He'll dig in and send you an update by the end of the week."

Just like that, I was a software architect. The rush of power. The anticipation of career advancement. I am an architect! Soon a slight feeling of dread set in. I am an architect. Now what do I do? How is being a software architect different from being a software engineer?

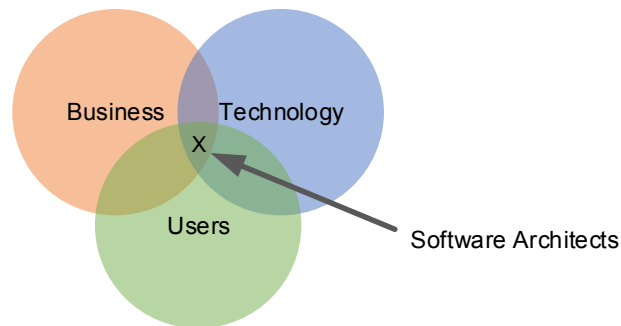
Software architects have a number of responsibilities in addition to programming. They define the problem from an engineering perspective. They divide the software system into implementable chunks, but also keep an eye on the big picture to ensure the system still works as a consistent whole. Architects decide trade-offs among quality attributes and manage the inevitable growth of technical debt. Perhaps above all, architects develop their team's architecture skills, because they know the best teams are filled with architects.

In this chapter, you'll learn what architects do. You'll also learn why knowing about software architecture will make you a better programmer and technical leader. You'll also learn how to get started on the path to becoming a software architect in your professional career.

## What Software Architects Do

Software architects are in a unique position on the team. They aren't project managers, but architects decide when and how software is delivered. They aren't product managers, but architects make sure the software meets its business goals. They write code, but architects design more than only algorithms and

code. Software architects have a distinct set of responsibilities and are seemingly at the center of everything.



Most of us start our software careers focusing purely on technology. Knowing how to program, design efficient algorithms, test that everything works, and deploy software are all essential skills for software architects too. Growing from programmer to software architect requires you to accept some new responsibilities.

## Define the Problem from an Engineering Perspective

Software architecture design is a human-centered design discipline. Everyone with a stake in the software can help you understand what is expected of it. Software architects work with product managers, project managers, and other stakeholders to define business goals and requirements for the software to be built.

On many teams, product managers define the features. Features are great, but there is another kind of requirement called a *quality attribute* that architects care the most about (see [Reason about Quality Attributes \(and Other System Properties\)](#), on page 10). In addition to defining quality attributes for the system, architects keep an eye out for design constraints and features that might force the architecture down a specific path.

Defining the problem with the architecture in mind ensures you can build a system everyone wants. You'll learn how architects approach requirements in [Chapter 5, Dig for Architecturally](#), on page 49.

## Partition the System and Assign Responsibilities

Have you ever watched little kids play soccer? The only tyke to stay in position is the goalie, who remains glued to their team's goal while the other children form a glob of kicking feet and chase the ball from one end of the field to the other. It's adorable. Once the kids grow up a bit, they learn to

play set positions. Playing positions is important since it lets the coach create a game strategy, which increases the team's likelihood of scoring.

Some software systems are designed like a children's soccer team: one great big clump of software chasing after a release. Software development, similar to soccer games, goes smoother when the software is divided into pieces and each piece is assigned a responsibility, a position to play.

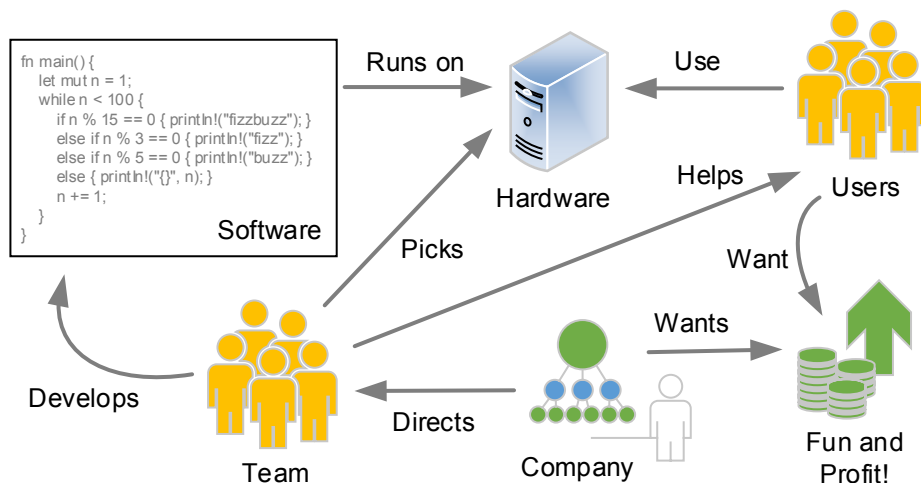
Architects partition (a fancy word for *divide into pieces*) the software system so they can develop a strategy for achieving quality attributes and other system requirements. For example, you might assign functional responsibilities by designing one component to register users and another to identify pictures of cats. Or you could assign different teams to develop different modules. Or you might split things that read data from things that write data so the software system will be more reliable, available, and scalable.

Partitioning a system is important not just because it lets you develop a strategy for achieving quality attributes. Smaller things are easier to reason about, easier to test, and easier to design. Of course, since you broke the system into pieces, you'll also have to make sure everything can come back together again.

## Keep an Eye on the Bigger Picture

Every software system lives in the context of a bigger world. The world in which software lives includes the users who interact with it, the team who builds it, the hardware it runs on, and even the purpose for developing the software in the first place. Ideally, the architecture lives harmoniously within this broader context.

Software always lives in the **context** of a **system**.



Thinking about the system as a whole means architects deal with more than just technology. People, processes, business needs, and many other technical and nontechnical factors play a part in the final software system. Even simple design decisions can have far-reaching consequences. Architects must look beyond a small neighborhood of design decisions and think about the system as a whole.

Software design is a constant struggle to find the right balance between the things you want and the reality you must accept. This means you must think about and make trade-offs.

### Decide Trade-offs among Quality Attributes

Say high availability is an important quality attribute for your stakeholders and you need your software to respond to 99.9 percent of requests. One way to promote availability is to introduce redundant elements. Designing for this is simple, but there's a catch. You now must purchase twice the hardware, which doubles your costs. In this case, you traded costs to get higher availability.

It is common in software development to give up something you want to get something you need. Architects identify the trade-offs and work with stakeholders to decide which compromises make the most sense.

Software systems are never partitioned perfectly. You'll make compromises. You'll make mistakes. As you build the system, you'll introduce technical debt into the architecture.

### Manage Technical Debt

Software architects know the details about how the system is partitioned. They keep an eye on the big picture and guide how everything comes together. They also connect technology decisions with business needs. Knowing all this puts architects in the perfect position to manage technical debt.

*Technical debt* is the gap between your software system's current design and the design you need it to have so you can continue to deliver value. You can measure the amount of technical debt by estimating the effort required to close that gap. All software has technical debt. Technical debt is an inevitable byproduct of success. The best software development teams use technical debt strategically to ship faster and regularly pay debt down so they can continue shipping value over time.

Architects make technical debt visible and help stakeholders decide which actions to take to manage it.

## Grow the Team's Architecture Skills

Software architects are teachers and mentors for their teams. There is no use in designing an awesome architecture that nobody can understand. As the architecture expert on your team, it is your responsibility to share your knowledge with your team so they can successfully develop an amazing software system.

Architects teach design skills and architecture concepts just-in-time. To pass on your knowledge, you'll pair design with teammates, create documents that educate and inform, and share constructive criticism. Perhaps the most important thing you can do to grow the team's architecture skills is to include them in the design process. Architecture design is a social activity. Skills development is crucial to your team's success.

Now you know what architects do, but we haven't defined what we mean by software architecture yet. Let's do something about that now.

## What Is Software Architecture?

A system's *software architecture* is the set of significant design decisions about how the software is organized to promote desired quality attributes and other properties.

A design decision might be significant for any number of reasons. It might represent a point of no return or influence quality attributes, schedule, or costs. A significant decision might be one that affects many people or forces other software systems to change. In any case, significant design decisions are costly to change later if you get them wrong.

To *promote* a quality attribute means to encourage it to appear in the software system. When the architecture is well organized, it will boost the quality attributes stakeholders want and downplay or eliminate the quality attributes stakeholders don't want. Architecture can promote other properties too. For example, the right architecture for the job will let you ship on time, on budget, and without requiring too much overtime.

## Define the Essential Structures

A skyscraper has a foundation and frames. A body has bones. Software has *structures*. A structure defines how a software system is arranged. Structures are in the code you write, the software you run, and even your collaborations with other people.

To create a structure, take any *element* and connect it to another element using a *relation*. Think of elements and relations like the bricks and mortar of software. The bread and peanut butter. The duct tape and...well, you get the idea. Elements are the fundamental building blocks of software. Relations describe how elements work together to accomplish some task.

It's easy to design an architecture on paper with no bearing on reality. To avoid this trap, you'll build architectures using three types of elements and relations. *Software Architecture in Practice [BCK12]* defines these three types as *module*, *component and connector* (C&C for short), and *allocation*. To create a structure, combine elements and relations of the same type.

Here are some example elements and relations of each type.

|                         | Example Elements                                                                    | Example Relations                                          |
|-------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------|
| Module                  | class, package, layer, stored procedure, module, configuration file, database table | uses, allowed to use, depends on                           |
| Component and Connector | object, connection, thread, process, tier, filter                                   | call, subscribe, pipe, publish, return                     |
| Allocation              | server, sensor, laptop, load balancer, team, Owen (a person), Docker container      | runs in or on, responsible for, develops, stores, pays for |

*Module* structures exist at design time. You interact with module structures when you write code. Module structures live on the file system and stick around even when the software is not running.

*Component and connector* structures come into existence at runtime. At runtime, components can create connections to other components, spawn new processes, and instantiate new objects. Unlike module structures, C&C structures cease to exist when the system is not running. You might only know a C&C structure existed from the artifacts it left behind, such as a log file or database entry.

*Allocation* structures are created by showing how module and C&C elements correspond with each other and the physical elements that exist in real life. Allocation structures are sometimes called *mapping structures* since they show how different elements map to one another. Does an element run on the client machine or the server? Which teams are building which parts of the system? Allocation structures help us answer questions like these.



**Joe asks:**

## Are Components and Modules Different Things?

In your software development career you may have heard the words module and component used interchangeably and in different contexts. Technically a component is a different concept from a module. A *module* refers to a design time element whereas a *component* is a runtime idea.

Sometimes this precision in language is important. Using a term with specific meaning to describe something general can create confusion. Anytime you want to describe a generic building block of an architecture instead of using component or module, use the word *element*.

All that said, arguing about semantics is not the best way to get your ideas across. While I encourage you to use proper and precise terminology, your ideas will sometimes get better mileage by adapting your language so others understand you.

Different kinds of structures are useful for thinking about different properties you want in your system. For example, you can think about testability and maintainability using a module structure. A C&C structure helps us think about runtime concerns such as availability or performance. You also might know there's a gap in our understanding if you see mixed structures such as a static element using a dynamic relation.

Structures determine the shape of our system. The shape is important since it decides the quality attributes and other properties your users will experience. In the next section you'll see how to use structures to reason about quality attributes, but first, it's time get your hands dirty with a quick exercise.



### Get Your Hands Dirty: Elements, Relations, and Structures

Find a few teammates from a recent project. Working alone, list or sketch module, component and connector, and allocation structures from that project. Share your lists with one another. How do they compare? Are there structures your teammates identified that you didn't? Discuss the similarities and differences in the structures different teammates identified.

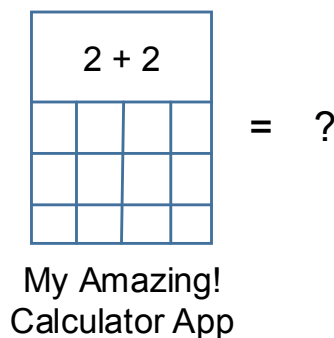
Here are some things to think about:

- Be specific when naming the elements. Don't forget about the relations!
- Think about the module structures: What methods or classes are used? Do the classes live in different packages or namespaces? What dependencies are included in package managers or build scripts?

- Think about the C&C structures: Does the software interact with other processes or systems at runtime? Who calls the system and how does it change in response?
- Think about the allocation structures: Who is responsible for building different parts of the software? How is the software deployed?

### Reason about Quality Attributes (and Other System Properties)

Say you're building a calculator app and you want to add two numbers together. Sounds easy, right?

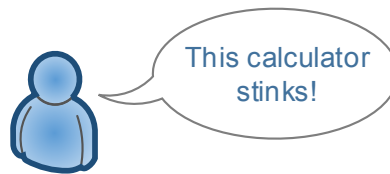


#### What if...

It takes 10 minutes to get an answer?

Sometimes you get the wrong answer?

You have to rewrite it to enable subtraction?



Wait. Did you want a calculator that adds two numbers and is fast, reliable, scalable, and maintainable? Why didn't you say so! If we hadn't asked about these quality attributes, we might have designed the wrong system altogether.

A *quality attribute* is any externally visible characteristic by which stakeholders judge a software system's goodness. Some examples include scalability, availability, maintainability, and testability. You experience quality attributes when you interact with the software.

When you choose an architectural structure, you are choosing the quality attributes you want to be promoted in the software system. Thinking about software architecture makes sure you design a software system that supports the quality attributes you want within the context of all the other concerns vying for attention.

Quality attributes make software unique. The circumstances of every system are different—different team, different budget, different market conditions,



even different technology trends. As a result, no two architectures will ever be the same even if the feature sets are identical.

Up for the challenge? In the next section, you'll learn some strategies for becoming your team's architect.

## Become an Architect for Your Team

On some teams, *architect* is an official team role. On other teams, there is no explicit role and teammates share the architect's responsibilities. Some teams say they don't have an architect, but if you look closely, someone is fulfilling the architect's duties without realizing it.

Architects are leaders, but being a *software architect* also implies a person who thinks about software design in a certain way. No matter what the title on your business card reads (mine still reads *software engineer*, my choice), you can be a software architect. Every team has at least one architect. The best teams have several.

If your team doesn't have an architect, congratulations, you've got the job! You don't need permission to inject architectural thinking into your team's design discussions. Start asking questions about quality attributes. Point out when the team makes trade-offs. Volunteer to write up design decisions and begin accepting more architecture design responsibilities.

If your team already has an architect, then ask that person how you can help. When possible, work closely with your architect and take advantage of every learning opportunity you can. Developing a software system is a big job. The more people who pay attention to the details, the greater your chance of success. Every team should be so lucky as to have many knowledgeable software architects!

## Make the Move from Programmer to Software Architect

An average software architect has developed three to five software systems with increased technical responsibility on each software system. Depending on the software you build, as your architecture responsibilities grow you may find you have less time for programming. This is normal, though software architects should never stop programming altogether.

To measure your growth from programmer to software architect, create a project portfolio. For every software system you build, no matter your role, briefly describe the software system and what you learned during your time developing it. This kind of reflective practice is essential for all technical leaders but especially software architects.

Here are some questions you should answer about each project in your portfolio:

- Who were the stakeholders and what were the primary business goals?
- What did the high-level solution look like?
- What technologies were involved?
- What were the biggest risks and how did you overcome them?
- If you could do it all over again, how would you do it differently?

Whether your goal is promotion or simply professional growth, be patient. You might have the chance to design a software system of meaningful complexity only every three to five years. If you are lucky, you will see between 8 and 15 software systems throughout your entire career. Be prepared to take advantage of architecting opportunities as they arise. Work with your teammates to give everyone a chance to grow their skills. I promise there is more than enough interesting architecture work for everyone!

Always remember, *software architect* is a way of thinking, not just a role on the team. When you're wearing your programmer hat, you'll make dozens of design decisions daily. Some of these decisions have architectural significance. Anyone who makes a decision that influences the structures of the software system becomes the *architect pro tempore*. It's up to you to make good decisions and uphold architectural integrity no matter what the title on your business card reads.

## Build Amazing Software

There are lots of things that have to go right when building a software system. Architecture connects them all together and provides a foundation for success. Here are six ways software architecture helps you in your quest to build spectacular software that your stakeholders will love:

### 1. Software architecture turns a big problem into smaller, more manageable problems.

Modern software systems are large and complex, and they have many moving pieces. The architecture precisely explains how to partition the system into smaller, bite-sized chunks while also ensuring the system as a whole is greater than the sum of its parts.

### 2. Software architecture shows people how to work together.

Software development is as much about human communication as it is technology. Software architecture describes how the whole system comes together, including the people who build it. When you know the architecture,

then you can see how people can collaborate to develop software. The larger the software system, the more important this becomes.

### 3. Software architecture provides a vocabulary for talking about complex ideas.

If I don't understand what you're talking about, then we won't be able to collaborate. Instead of spending all our time inventing vocabulary and concepts, we can use the essential concepts and core vocabulary of architecture as the starting basis for collaboration. Now we can spend our time solving our users' real problems.

### 4. Software architecture looks beyond features and functionality.

Features and functionality are important, but they are not the only thing that determines whether or not software is awesome. When designing architecture, you'll consider not only the features but also costs, constraints, schedules, risk, the ability of the team to deliver, and most importantly quality attributes—things like scalability, availability, performance, and maintainability.

### 5. Software architecture helps you avoid costly mistakes.

In *Who Needs an Architect?* [Fow03], Martin Fowler defines software architecture as "...the important stuff. Whatever that is." The important stuff is nearly always what we think will be difficult to change without significantly increasing complexity. Grady Booch echoes Fowler's sentiment by defining architecture as the "...significant design decisions (where significant is measured by the cost of change)."<sup>1</sup> Software architects are not omniscient, but designing an architecture will help you discover the challenging (and interesting) parts of the problem that might cause big trouble later.

### 6. Software architecture enables agility.

Your software should respond to change like water, by bending around obstacles with ease. If software is like water, able to take any shape, then software architecture is the container that holds it. That container can be rigid like a box or flexible like a plastic bag. It can be thick and heavy or lightweight. Without an architecture, software, like water, follows the path of least resistance and sprawls uncontrollably. A software system's architecture provides the structure within which change is possible.

We'll expand on these ideas throughout the remainder of the book.

---

1. Grady Booch. *Abstracting the Unknown*. SATURN 2016. <http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=454315>

## Case Study: Project Lionheart

As we cover new ideas in each chapter, we'll apply them to a case study, Project Lionheart. The case study is based on a real system, but the names and situations have been changed for teaching and legal purposes.

### Design an Architecture to Solve This Problem

The City of Springfield is facing budget shortfalls and needs to cut costs. Mayor Jean Claude van Damme (no relation to the action hero) has hired our team to streamline the city's Office of Management and Budget (OMB).

When a city employee needs to purchase something for more than a few thousand dollars, the OMB issues a Request for Proposals (RFP) in the local newspaper. Businesses bid on the RFPs and the OMB awards a contract based on the competitiveness of the bid and other factors. The OMB monitors more than 500 active contracts and RFPs for everything from toilet paper to medical supplies to basketballs. The OMB manages all this data in spreadsheets.

Mayor van Damme hopes modernizing the OMB will improve a few strategic areas.

- Over half of all RFPs have a single bid. The city is potentially overpaying for lower-quality services.
- Finalizing a contract takes months. Many businesses get lost in the multistep process.
- Publishing a new RFP takes up to 6 weeks. This process must be faster.

Throughout Part II, we'll flesh out this case study and work together to design a plausible architecture to solve some of these problems.

## Next Up

Software architects are responsible for quite a lot. Designing interesting, complex software systems and working with different people feels good and is well worth the effort. Becoming a software architect is not an overnight journey. If you focus on the architect's core responsibilities and do your best to apply the architectural fundamentals, mainly selecting structures to promote desired quality attributes, then you'll do great.

In this chapter, you learned what architecture is and what architects do. In the next chapter, you'll learn how to use design thinking to figure out what should go into the architecture.